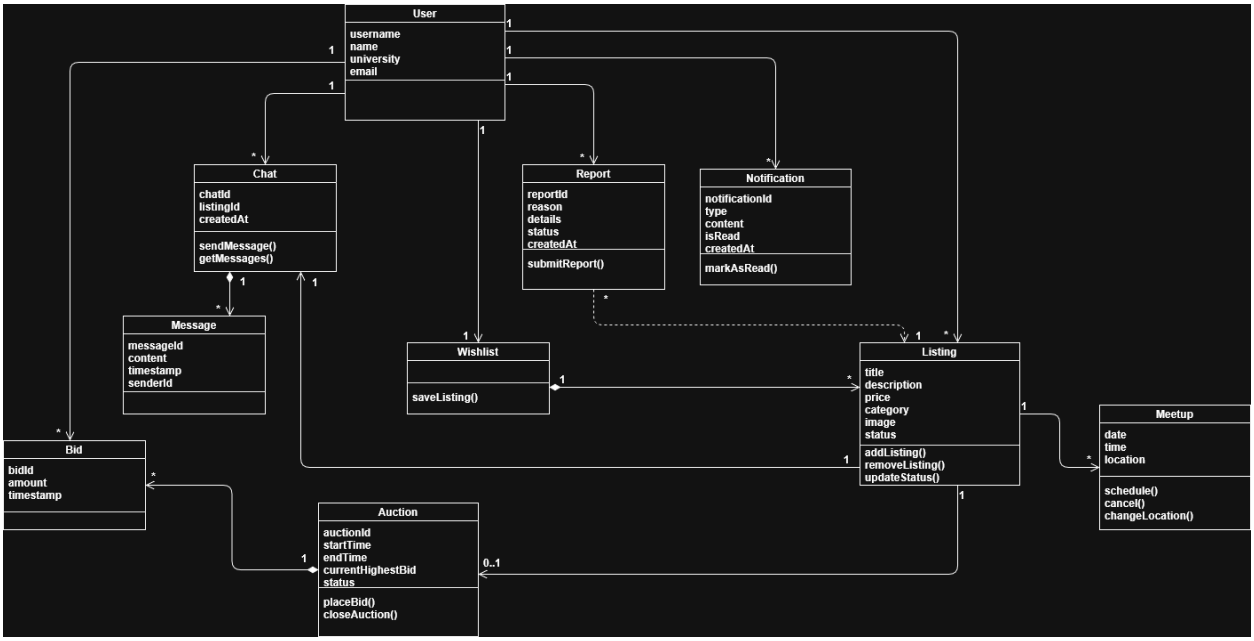
Sprint 3 — System Architecture Document

1. Revised Analysis Object Model (UML class diagram):



2. Design goals:

1. Performance:

- Metric/measurement: 95% of user actions (page loads, listing views, chat messages) must complete in under 2 seconds under a load of 500 concurrent users.
- Justification: fast response times are critical for user satisfaction and engagement. Marketplace users expect near instant feedback when browsing listings or messaging sellers, and delays may discourage usage.

2. Security and Privacy:

- Metric/measurement: 100% of sensitive data (user credentials, messages, personal info) must be encrypted with zero unauthorized access incidents.
- Justification: the system handles sensitive personal and transactional data. Strong encryption and strict access control are essential to protect users and maintain trust within the campus community.

3. Availability:

- Metric/measurement: The system must maintain >= 99.5% uptime during the academic semester, excluding schedules maintenance.
- Justification: Students rely on the platform at all times to buy, sell, and coordinate exchanges. High availability ensures reliability and consistent access, especially during peak usage periods.

4. Usability:

- Metric/measurement: At least 90% of first-time users must be able to complete key tasks (find an item, create a listing, send a message) within 3 minutes, with no external help.
- Justification: a simple and intuitive interface reduces the learning curve and encourages adoption, especially for new users who may only use the platform occasionally.

5. Fault Tolerance:

- Metric/measurement: At least 80% of the system must be available to users in the case of a component failing or having an error.
- Justification: If a component fails such as the Listing service, users should still be able to interact and use other components in the application in the meantime, so it isn't a complete shutdown.

6. Robustness

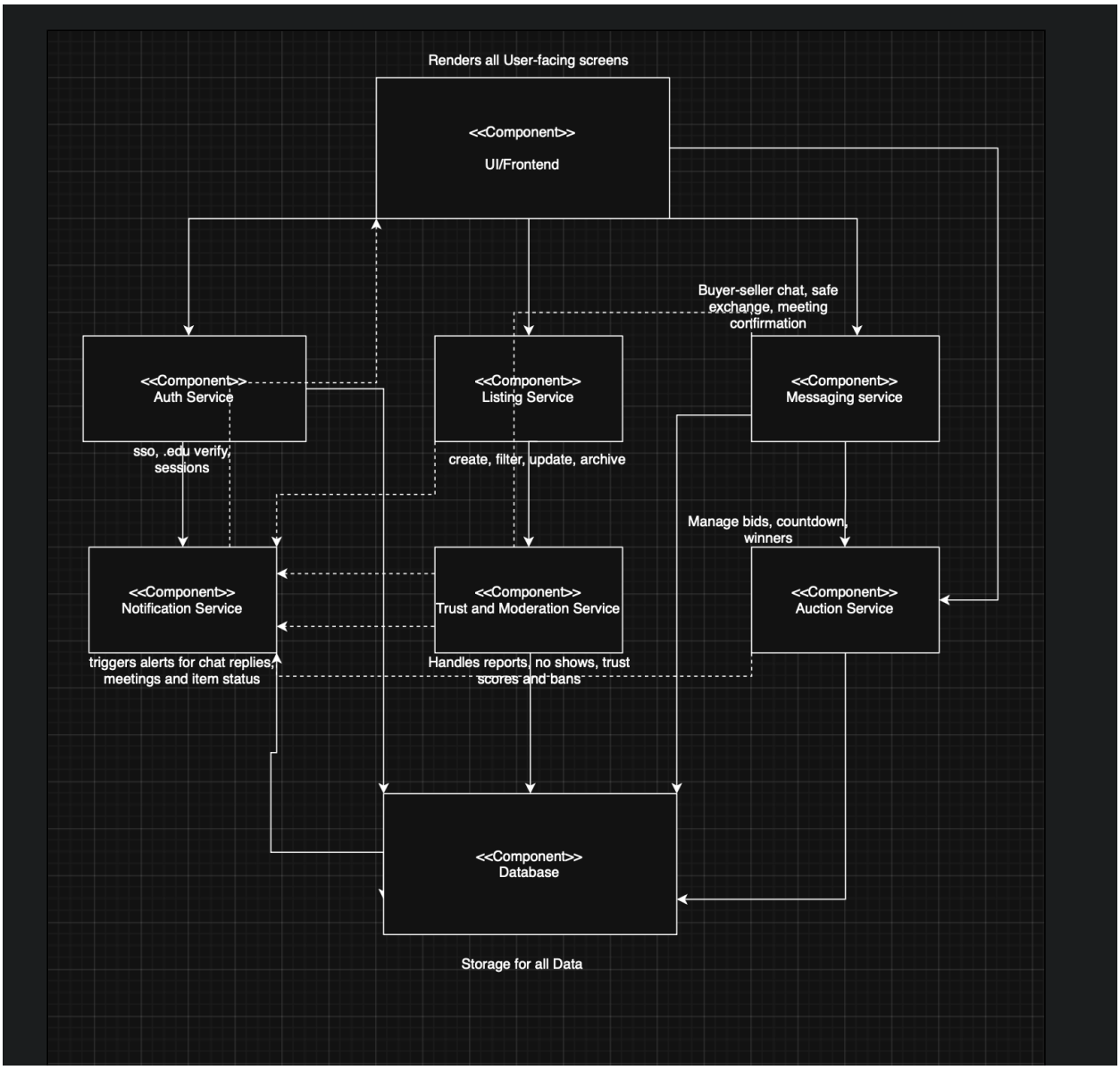
- Metric/measurement: The system must handle 100% of invalid user inputs without crashing or failing, providing error messages to the user to identify the invalid input.
- Justification: In the case of a user inputting invalid information, the system should not fail or crash, it should simply provide a descriptive error message indicating what went wrong, so users know what to input.

7. Reliability

- Metric/measurement: The system must maintain >= 99% sucess rate on all actions the system performs.
- Justification: Users must expect features to work and be successful in order to utilize the system to its full degree. Actual behavior must not deviate from specified behavior frequently.

3. Subsystem Decomposition:

UML component diagram



UI / Frontend

- Responsibilities: Renders all user-facing screens
- Connects To: All services

Authentication Service

- Responsibilities: SSO login, .edu verification, session management

- Connects To: UI, Database

Listing Service

- Responsibilities: Create, filter, update, and archive listings
- Connects To: UI, Database, Notification Service

Messaging & Scheduling Service

- Responsibilities: Buyer-seller chat, Safe Exchange Zone proposals, meeting confirmation
- Connects To: UI, Notification Service, Database

Notification Service

- Responsibilities: Triggers alerts for chat replies, meeting updates, and item status changes
- Connects To: All services

Trust & Moderation Service

- Responsibilities: Handles reports, no-shows, trust scores, and bans
- Connects To: UI, Database, Notification Service

Auction Service

- Responsibilities: Manages bids, countdown timer, and winner selection
- Connects To: UI, Database, Notification Service

Database

- Responsibilities: Persistent storage for all system data
- Connects To: All services

Subsystem Design Rationale:

The subsystem decomposition follows a separation of concerns principle, where each subsystem encapsulates a distinct functional responsibility of the marketplace system.

- High-cohesion design: Each subsystem (e.g., Listing Service, Messaging Service, Auction Service) is responsible for a single core domain functionality, minimizing internal complexity.
- Low coupling: Subsystems interact through clearly defined interfaces, reducing dependencies, and enabling independent updates.
- Scalability consideration: Services such as Auction and Messaging are separated due to their potential high concurrency requirements.
- Alignment with design goals: Security-sensitive operations are isolated in the Authentication Service, while performance-critical features are distributed across independent services.

This decomposition ensures the system remains modular, maintainable, and scalable as user demand grows.

4. Architectural Reasoning & Design Decisions:

Our system utilizes a **Layered Architectural Style** combined with a **Service-Oriented Decomposition** to ensure the system remains testable against our measurable design goals.

- **Supports Scalability for Auctions:** Isolating the Auction Service in the Application layer allows the system to handle high-concurrency bidding and 30-day deadlines (FR17) without degrading the performance of the core marketplace.
- **Enforces Trust & Security:** A closed layering approach ensures all UI requests pass through the Authentication Service before accessing data, supporting our goal of 100% encryption and zero unauthorized access.
- **Decouples Moderation Logic:** Separating the UI from the Trust & Moderation Service allows us to update "No-Show" penalty logic (FR14) and account deletion flows (FR15) independently without affecting the frontend presentation.
- **Centralizes Communications:** A shared Notification Service within the Application layer manages all alerts for outbids and meeting updates, ensuring a consistent user experience and supporting our <3-minute task completion goal.
- **Ensures Data Integrity:** A centralized Database layer accessed only through a dedicated Persistence layer provides the consistency needed for our 99.5% uptime goal while securely anonymizing data during account closures.

- **Service isolation for concurrency**: Auction and Messaging services are designed as independent components to handle high-frequency operations without impacting core listing performance.
- **Interface-based communication**: All services communicate through defined service interfaces rather than direct database access, improving extensibility, and enabling future API integration.
- **Centralized authentication gateway**: All incoming requests must pass through the Authentication Service before reaching other subsystems, enforcing a consistent security boundary across the system.

Roles:

Britney Lu (SM): Refined the initial draft of the Architectural Reasoning in Section 4

Emily Bartell (PO): Produced design goals 1 through 4 in Section 2 relating to performance, security, availability, and usability.

Thomas Jones (Dev): Produced Design goals 5-7 in section 2 relating to fault tolerance, robustness, reliability.

Senay Yemane (Dev):

Xiangyu Li (Dev): Refined subsystem decomposition rationale in Section 3, and enhanced architectural design decisions in Section 4.

Jack Drabenstadt (Dev): Revised the Analysis Object Model based on Sprint 2 feedback, drafted the subsystem decomposition for Section 3, and drafted the architectural reasoning for Section 4.